

RAG System Observability Manual

This document defines the comprehensive telemetry layout, state-marker indices, action tracking logs, and human-in-the-loop (HITL) audit streams of the document-agnostic multimodal RAG toolkit.

Symmetrically, this guide serves as the definitive reference for understanding active state tracking, execution monitoring, and debugging.

1. System State Markers

State markers are lightweight files that represent the operational status of the guided pipeline steps. Symmetrically, they are stored inside the active project directory under `logs/` (except `.active_project`, which is stored in the repository root):

`.active_project`

- **Location:** Repository Root (`/path/to/IndyRAG/.active_project`)
- **Trigger Command:** `rag checkin <project-name>` (Step 0)
- **Role & Purpose:** Holds the resolved absolute file path to the active project workspace. Symmetrically, this command also performs dynamic directory scaffolding (settling `logs/`, `crops/`, and `library/` folders inside the targeted workspace) immediately upon check-in to prevent repository-root folder pollution. Symmetrically, all downstream pipeline commands parse this pointer on boot to automatically locate the correct workspace.

`.init.ok`

- **Location:** `logs/.init.ok`
- **Trigger Command:** `rag init` (Step 1)
- **Role & Purpose:** Touched upon a successful environment capabilities audit and platform validation (folder scaffolding is handled earlier during check-in).

`.index_build.ok`

- **Location:** `logs/.index_build.ok`
- **Trigger Command:** `rag build-index` (Step 14)

- **Role & Purpose:** Touched upon successfully constructing the full-text keyword search and semantic vector databases.

.verify.ok

- **Location:** `logs/.verify.ok`
- **Trigger Command:** `rag verify` (Step 15)
- **Role & Purpose:** Touched upon passing all programmatic integrity checks (SHA-256 hashes, crop presence, and approval audits). Unlinked on new verification boots to prevent stale exports.

.warmup.ok

- **Location:** `logs/.warmup.ok`
- **Trigger Command:** `rag query` (Step 17)
- **Role & Purpose:** Touched upon executing either a single query or a batch processing query to register model warmup states.

.chat.ok

- **Location:** `logs/.chat.ok`
- **Trigger Command:** `rag chat` (Step 18)
- **Role & Purpose:** Touched upon launching the interactive conversational session successfully.

2. Agent Action Telemetry

Action telemetry logs are structured JSON-Lines streams that capture the operational reasoning, tool allocations, and token weights of our background multi-turn agents. Symmetrically, they are stored inside `logs/` :

macro-images.jsonl

- **Location:** `logs/macro-images.jsonl`
- **Responsible Agent:** Pass 1 Discovery Agent (Step 5)
- **Logged Events:** Captures the full session telemetry of our multi-turn layout extraction agent, recording tool inputs, coordinates, token payloads, and prompt milestones.

micro-images.jsonl

- **Location:** `logs/micro-images.jsonl`
- **Responsible Agent:** Pass 2 Subdivision Agent (Step 7)
- **Logged Events:** Captures the session telemetry of our multi-turn granular detail subdivisions agent.

macro-recrop.jsonl

- **Location:** `logs/macro-recrop.jsonl`
- **Responsible Agent:** Re-Crop Agent (Step 6)
- **Logged Events:** Captures the full session telemetry of our multi-turn layout correction agent, recording tool inputs, coordinates, token payloads, and prompt milestones during macro correction loops.

micro-recrop.jsonl

- **Location:** `logs/micro-recrop.jsonl`
- **Responsible Agent:** Re-Crop Agent (Step 8)
- **Logged Events:** Captures the session telemetry of our multi-turn granular detail correction agent during micro correction loops.

3. Verification & Lifecycle Tracking

Lifecycle logs record automated, round-level approvals, programmatic checks, and permanent verification failures. Symmetrically, they are stored inside `logs/` :

macro-verify.jsonl

- **Location:** `logs/macro-verify.jsonl`
- **Responsible Process:** Verification Loop (Step 6)
- **Logged Events:** Captures the programmatic checks, Pass/Fail results, and JSON response payloads of the 1-shot macro-level Flash verifier.

micro-verify.jsonl

- **Location:** `logs/micro-verify.jsonl`
- **Responsible Process:** Verification Loop (Step 8)

- **Logged Events:** Captures the programmatic checks, Pass/Fail results, and JSON response payloads of the 1-shot micro-level Flash verifier.

items_promoted.jsonl

- **Location:** `logs/items_promoted.jsonl`
- **Responsible Process:** Verification Loop (Steps 6 & 8)
- **Logged Events:** Records a JSON row whenever a candidate draft passes Flash verification and is promoted to the canonical regions database.
- **Schema Fields:** Includes timestamp (`ts`), `doc_id` , `page` , `label` , and the exact `round_verified` index.

items_discarded.jsonl

- **Location:** `logs/items_discarded.jsonl`
- **Responsible Process:** Verification Loop (Steps 6 & 8)
- **Logged Events:** Records a JSON row whenever a candidate draft permanently fails all verification rounds or is unrecoverable, resulting in its removal from the drafts manifest.
- **Schema Fields:** Includes timestamp (`ts`), `doc_id` , `page` , `label` , and the exact `discard_reason` (e.g., failed verification round limits or agent errors).

4. Human-In-The-Loop (HITL) Audit Trails

Audit trails record manual operator actions and region rejections during human verification gates inside `logs/` :

assets_rejected.jsonl

- **Location:** `logs/assets_rejected.jsonl`
 - **Responsible Process:** HITL review controllers (Steps 9 & 13)
 - **Logged Events:** Symmetrically logs transaction rows detailing manual operator rejections of visual regions or bound assets.
-

5. Console & Package Telemetry

Standard diagnostic printouts and third-party warnings are written to their respective logs inside `logs/` :

`<command>.log`

- **Location:** `logs/<command>.log` (e.g., `logs/ingest.log` , `logs/caption.log`)
- **Responsible Process:** Observe logging wrapper (`src/utils/observe.py`)
- **Logged Events:** Symmetrically captures standard CLI stdout printouts, progress bars, and execution metrics on every run.

`hf.log`

- **Location:** `logs/hf.log`
- **Responsible Process:** Quiet environment manager (`src/utils/hf_quiet.py`)
- **Logged Events:** Redirects and captures all HuggingFace hub progress bars, download warnings, and transformers logs, keeping the terminal output clean.